MICROGOALS - Secure Goal Management Backend API

Backend Development Project Documentation (Node.js, Express.js, Mongodb, Thunder Client)

Team Members:

- **Sairunisha S M** – Team Leader
- **Swetha R**
- **Lokeshwari P**
- **Yuvashri R**

1. Project Overview

MicroGoals is a secure backend RESTful API built with **Node.js**, **Express.js**, and **MongoDB**. It allows users to register, log in, and manage their personal goals using full CRUD (Create, Read, Update, Delete) operations.

- **Authentication:** JWT-based
- **Architecture:** MVC
- **Security:** Protected routes, hashed passwords, JWT verification
- **Database:** MongoDB (local or Atlas)

Objective: To provide a secure, scalable backend system for managing user-specific digital goals.

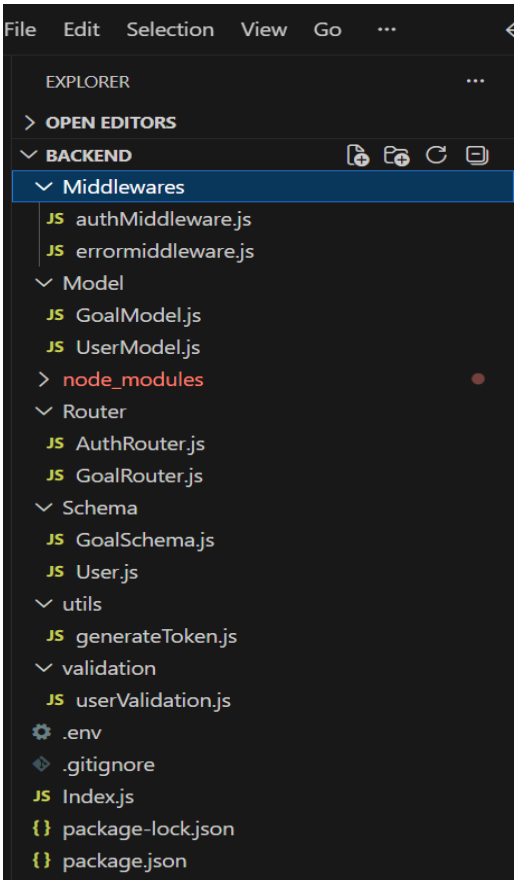
2. Features

1. User Registration
2. Secure Login with JWT
3. Create Goal
4. Update Goal
5. Delete Goal
6. View User-specific Goals
7. **View All Goals for Logged-in User** (`/all`) *(Additional feature added)*
8. Protected Routes
9. Centralized Error Handling

The `/all` route was added to allow users to fetch **all their goals at once**, giving a clear, professional overview of tasks. This is especially useful in demos to show a full array of goals.

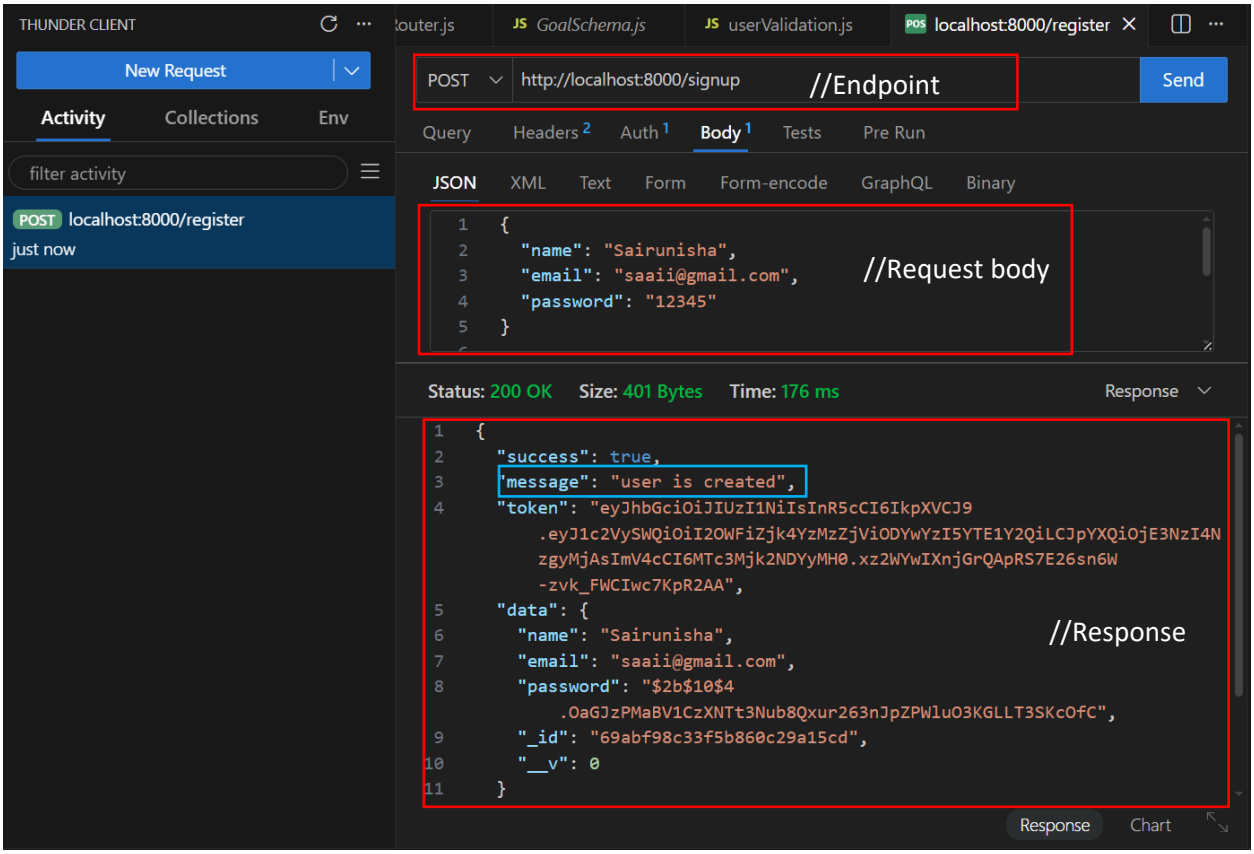
3. Project Architecture

Folder Structure:



4. API Endpoints & Thunder Client Screenshots

I. User Registration:



Endpoint allows a new user to register on the MicroGoals platform. It accepts the user's **name, email, and password**, hashes the password securely using **bcrypt**, and creates a new user entry in the MongoDB database. Upon successful registration, a **JWT token** is generated and returned. This token will be used to authenticate future requests to protected routes.

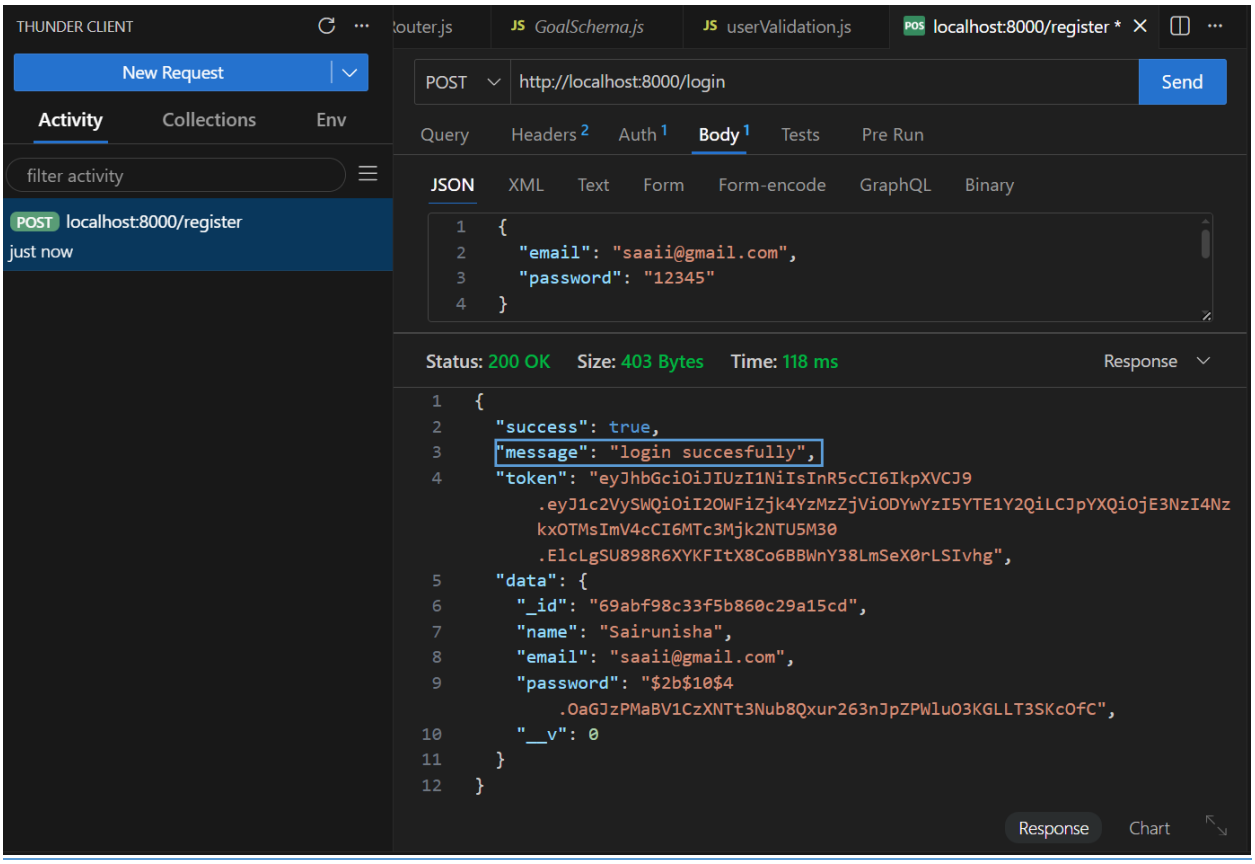
success: Indicates that the operation was successful.

message: Confirms that the user has been created.

token: The JWT token generated for this user, required for authenticating future requests.

data: Shows the user object stored in the database. The `password` is hashed for security, so it is not stored in plain text. `_id` is the unique identifier for the user in MongoDB.

II. User Login:



Allows a registered user to log in using their **email and password**. If the credentials are correct, a **JWT token** is generated and returned. This token is required for accessing protected routes like creating or managing goals.

success: Indicates login was successful.

message: Confirms the login action.

token: JWT token for authenticated requests.

data: User details; the password is hashed for security.

This endpoint ensures **secure user authentication**. Without the JWT token returned here, users cannot access any protected routes like goal creation or updates.

III. Create goals:

POST http://localhost:8000/create Send

Query Headers 2 Auth 1 Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

```
{
 "title":"Complete assignment"
}
```

//Goal 1

Status: 200 OK Size: 148 Bytes Time: 73 ms Response

```
{
  "success": true,
  "data": {
    "title": "Complete assignment",
    "completed": false,
    "user": "69abf98c33f5b860c29a15cd",
    "_id": "69ac056633f5b860c29a15e6",
    "__v": 0
  }
}
```

POST http://localhost:8000/create Send

Query Headers 2 Auth 1 Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

```
{
 "title":"Go for a walk"
}
```

//Goal 2

Status: 200 OK Size: 142 Bytes Time: 62 ms Response

Copy

```
{
  "success": true,
  "data": {
    "title": "Go for a walk",
    "completed": false,
    "user": "69abf98c33f5b860c29a15cd",
    "_id": "69ac05ec33f5b860c29a15ea",
    "__v": 0
  }
}
```

Allows a logged-in user to create a new personal goal. Each goal is linked to the authenticated user via their user ID.

success: Confirms the goal was successfully created.

data: The goal object with details:

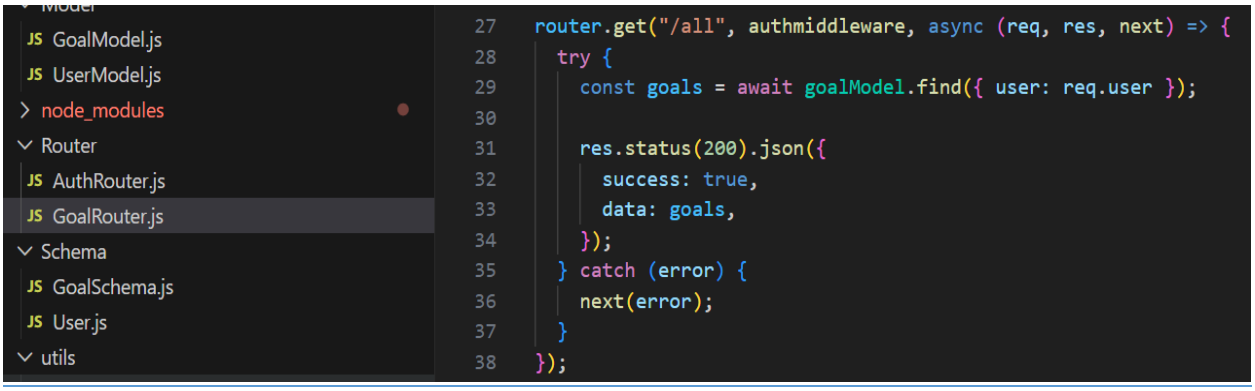
- _id: Unique identifier for the goal.

- title: The goal's title. Complete assignment and go for a walk.
- completed: Initially `false` until the user marks it done.
- user: The ID of the user who created the goal.

This endpoint demonstrates **user-specific goal creation**. Each goal is tied securely to its owner, allowing multiple goals to be stored per user. This is also the foundation for showing all of the goals together using the `/all` endpoint which I additionally added.

IV. Get All Goals (Additional Feature):

Required Code:



The `/all` route is an additional feature added to allow a logged-in user to retrieve **all their goals at once**. It is a **GET endpoint** protected by the `authmiddleware`, ensuring that only authenticated users with a valid JWT token can access it. When a request is made to `/all`, the route queries the `goalModel` collection in MongoDB using `find({ user: req.user })`, which fetches all goal documents where the `user` field matches the ID of the currently logged-in user. This returns an **array of all goals** belonging to that user, allowing multiple goals to be displayed in a single response. The API then responds with a JSON object containing `success: true` and the `data` array, which holds the list of goals. This setup ensures **user-specific data retrieval**, prevents access to other users' goals, and provides a professional, clean output that is ideal for demonstrations. If the user has no goals, it simply returns an empty array. Any errors encountered during the process are forwarded to the centralized error middleware, maintaining consistent error handling across the API.

GET http://localhost:8000/all Send

Query Headers 2 Auth 1 Body 1 Tests Pre Run

None Basic Bearer OAuth 2 NTLM AWS

Bearer Token

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWQiOiI2OWFiZDUzY2JmZDY5ZDhlMTc5MzBhNTciLCJpYXQiOjE3NzI4ODA4MjQsImV4cCI6MTc3Mjk2NzIyNH0.hNzAxzr-gfUaf6gChkxcl6P54Oc5MqoIKU2F9EyH5dg

GET http://localhost:8000/all Send

Query Headers 2 Auth 1 Body 1 Tests Pre Run

Status: 200 OK Size: 269 Bytes Time: 68 ms Response


```
1  {
2    "success": true,
3    "data": [
4      {
5        "_id": "69ac056633f5b860c29a15e6",
6        "title": "Complete assignment",
7        "completed": false,
8        "user": "69abf98c33f5b860c29a15cd",
9        "__v": 0
10     },
11     {
12       "_id": "69ac05ec33f5b860c29a15ea",
13       "title": "Go for a walk",
14       "completed": false,
15       "user": "69abf98c33f5b860c29a15cd",
16       "__v": 0
17     }
18   ]
19 }
```


Response Chart

Retrieves **all goals created by the logged-in user**. This is an additional feature that allows users to see their complete list of goals in one response, making the API more professional and user-friendly.

`success`: Confirms the API call was successful.

`data`: An array of all goals belonging to the logged-in user. Each object contains the goal's `_id`, `title`, `completed` status, and `user` reference. This endpoint demonstrates **multi-goal retrieval** in a single call, which is useful for dashboards or user overviews.

Here, this **additional `/all` feature** improves the usability of the application since users can now view all their goals at once.

V. Get Goal by ID:

GET http://localhost:8000/69ac05ec33f5b860c29a15ea Send

Query Headers 2 Auth 1 Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

Status: 200 OK Size: 142 Bytes Time: 81 ms Response

Copy


```
{
  "success": true,
  "data": {
    "_id": "69ac05ec33f5b860c29a15ea",
    "title": "Go for a walk",
    "completed": false,
    "user": "69abf98c33f5b860c29a15cd",
    "__v": 0
  }
}
```


Retrieves a specific goal of the logged-in user by its **unique ID**.

Only goals belonging to the authenticated user are returned.

Useful for viewing **specific goal details**.

NOTE: In completed column, it is displayed as work not done (“false”). In the next step that output will be changed.

VI. Update Goal Completion Status:

PUT http://localhost:8000/69ac05ec33f5b860c29a15ea Send

Query Headers 2 Auth 1 Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

```
{
 "completed": true
}
```

Status: 200 OK Size: 50 Bytes Time: 75 ms Response

Copy

```
{
  "success": true,
  "message": "the goal is completed"
}
```

GET http://localhost:8000/69ac05ec33f5b860c29a15ea Send

Query Headers 2 Auth 1 Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

```
{
 "completed": true
}
```

Status: 200 OK Size: 141 Bytes Time: 50 ms Response

Copy

```
{
  "success": true,
  "data": {
    "_id": "69ac05ec33f5b860c29a15ea",
    "title": "Go for a walk",
    "completed": true,
    "user": "69abf98c33f5b860c29a15cd",
    "__v": 0
  }
}
```

`success: true` indicates the request was processed successfully.

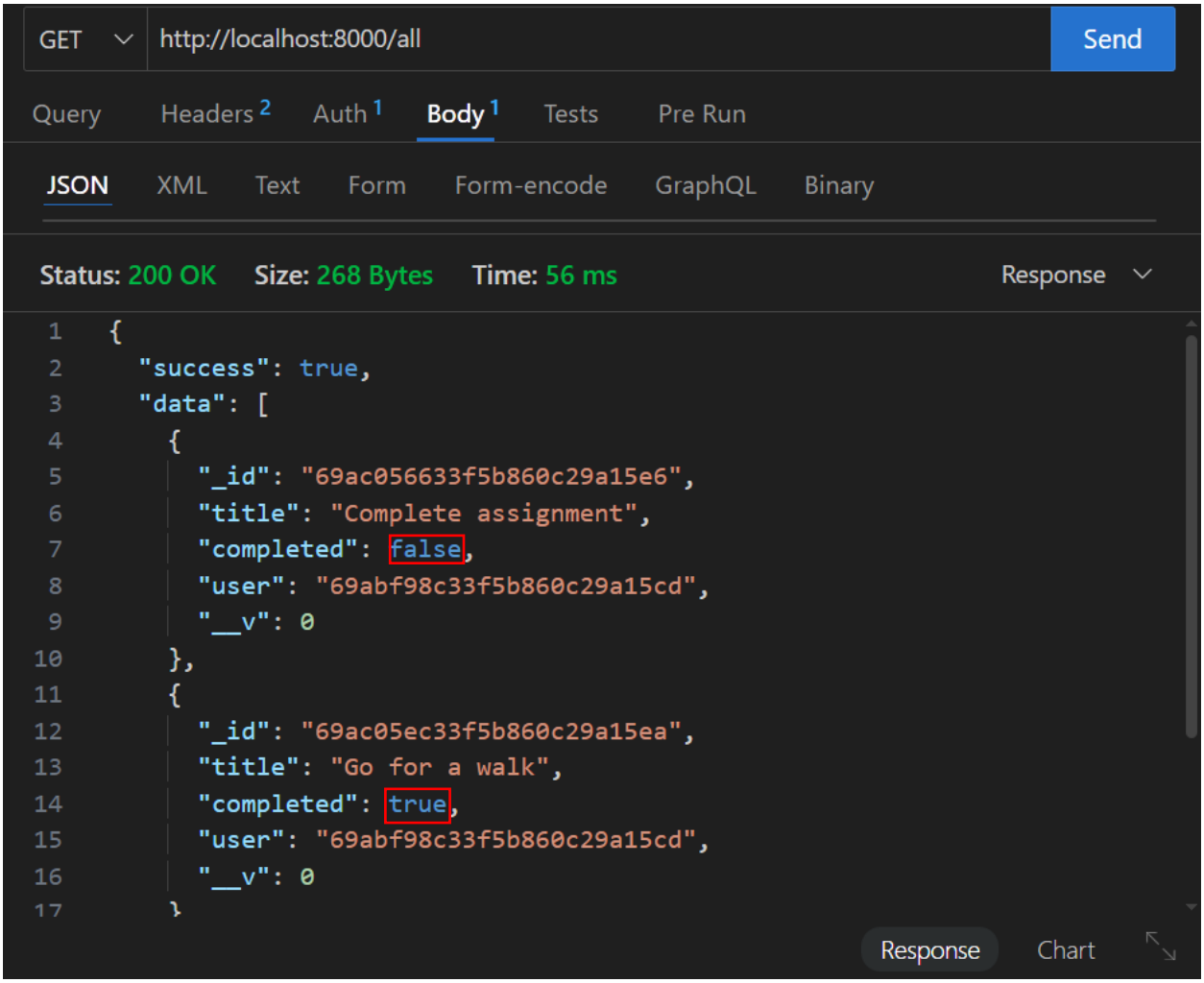
`message: "the goal is completed"` confirms the goal's status was updated.

`data` contains the updated goal object:

- `_id` is the unique identifier of the goal.
- `title` shows the goal name ("Go for a walk").
- `completed: true` confirms that the goal is now marked as done.
- `user` is the ID of the user who owns the goal, ensuring the update is tied to the correct user.

This output shows that the API correctly updates the goal's completion status for the logged-in user, and the change is reflected immediately in the database.

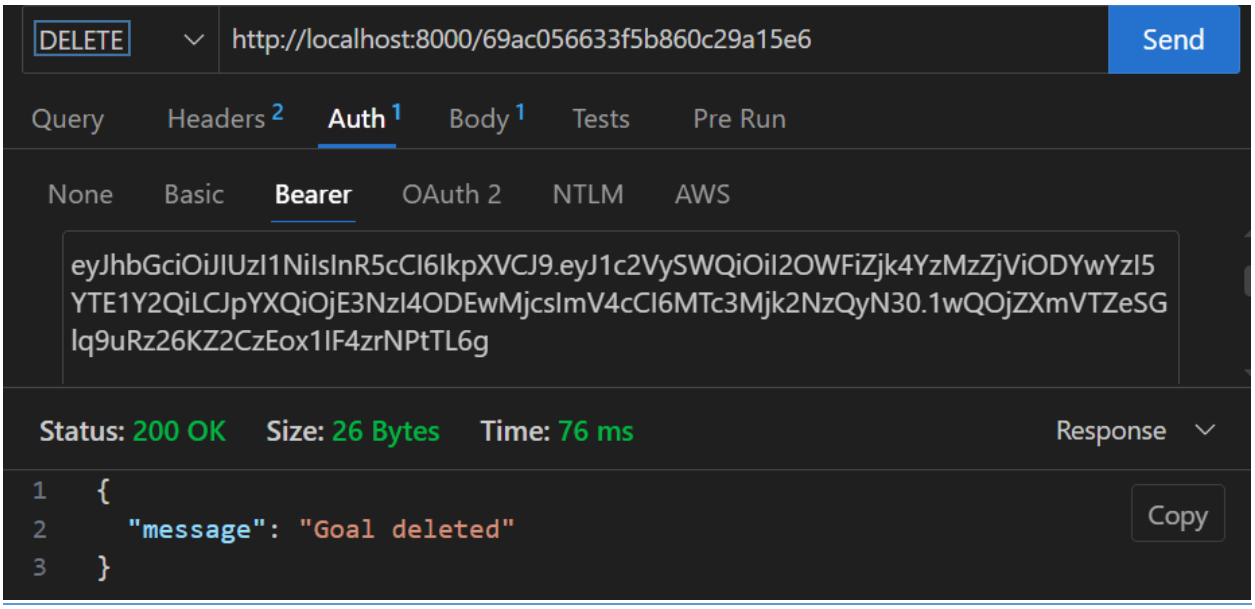
VII. Status check of all the goals:




```
{
  "success": true,
  "data": [
    {
      "_id": "69ac056633f5b860c29a15e6",
      "title": "Complete assignment",
      "completed": false,
      "user": "69abf98c33f5b860c29a15cd",
      "__v": 0
    },
    {
      "_id": "69ac05ec33f5b860c29a15ea",
      "title": "Go for a walk",
      "completed": true,
      "user": "69abf98c33f5b860c29a15cd",
      "__v": 0
    }
```

This endpoint provides a complete overview of all goals for a user, showing which tasks are completed and which are pending. In this example, “Complete assignment” is still pending (`false`), while “Go for a walk” has been completed (`true`). This makes it easy to track progress and demonstrates the functionality of the additional `/all` feature.

VIII. Delete Goal:



Allows a logged-in user to permanently delete one of their goals. Only the owner of the goal can perform this action.

- message: confirms the deletion action.

This endpoint demonstrates secure deletion functionality. Only authenticated users can delete their own goals, ensuring data safety and user-specific access control. In the example, after sending a DELETE request for a goal, it is removed permanently from the database, and subsequent requests for that goal by ID will return a "not found" error. This completes the CRUD operations for professional goal management.